

```
%% MATLAB Code: Filtering with FPGA Input and Graphical✓  
Plots
```

```
% Step 1: Define Thresholds and Vibration Classes (from✓  
provided image)
```

```
thresholds = [100, 200, 300, 500, 1000, 2000]; % Example✓  
threshold frequencies in Hz
```

```
classes      = {'Low', 'Medium', 'High', 'Higher', 'Very High',✓  
'Severe'};
```

```
% Step 2: Setup and Acquire Input Signal from FPGA (TTL✓  
interface via DAQ)
```

```
% This example uses MATLAB's Data Acquisition Toolbox for an✓  
NI device.
```

```
fs = 5000; % Sampling frequency in Hz  
acqDuration = 2; % Acquire 2 seconds of data
```

```
% Create a DAQ session (adjust the device ID 'Dev1' and✓  
channel as needed)
```

```
d = daq("ni");
```

```
d.Rate = fs;
```

```
% Add an analog input channel (modify 'ai0' and 'Voltage'✓  
based on your FPGA output setup)
```

```
ch = addinput(d, "Dev1", "ai0", "Voltage");
```

```
% Acquire data from the FPGA (TTL signals should be✓  
conditioned to be read as analog voltage)
```

```
disp('Acquiring data from FPGA...');
```

```
data = read(d, seconds(acqDuration));
```

```
% Data from the channel is in table format; convert to✓  
vector:
```

```
input_signal = data(:, 'ai0');
```

```
% Generate corresponding time vector for acquired samples
```

```
t = (0:length(input_signal)-1) / fs;

% Step 3: Apply a Notch Filter to Remove 50 Hz Interference
wo = 50/(fs/2);           % Normalized center frequency✓
for 50 Hz
bw = wo/35;               % Bandwidth for the notch filter
[b_notch, a_notch] = iirnotch(wo, bw);           % Design✓
the notch filter
signal_notched = filter(b_notch, a_notch, input_signal); %✓
Filter the signal

% Step 4: Apply a High-Pass Filter to Remove Low Frequency✓
Noise (below 10 Hz)
hp_cutoff = 10;           % Cutoff frequency for the high-✓
pass filter
[b_hp, a_hp] = butter(2, hp_cutoff/(fs/2), 'high'); % 2nd✓
order Butterworth high-pass filter
signal_hp = filter(b_hp, a_hp, signal_notched);    % Filter✓
the signal

% Step 5: Apply a Moving Average Filter to Smooth the Signal
windowSize = 50;          % Define the moving average✓
window size
ma_kernel = ones(1, windowSize) / windowSize;      % Create✓
a normalized moving average kernel
signal_smoothed = filter(ma_kernel, 1, signal_hp);  % Apply✓
the moving average filter

% Step 6: Compute the FFT to Determine the Dominant Frequency
N = length(signal_smoothed);           % Number of data✓
points
Y = fft(signal_smoothed);               % Compute FFT of the✓
smoothed signal
f = (0:N-1) * (fs/N);                   % Frequency vector✓
```

```
corresponding to FFT bins
power = abs(Y).^2; % Compute power✓
spectrum of the FFT

% Find the dominant frequency (only consider frequencies up✓
to Nyquist)
[~, index] = max(power(1:floor(N/2)));
dominant_freq = f(index);

% Step 7: Compare Dominant Frequency to Thresholds to✓
Classify the Vibration Level
class_idx = find(dominant_freq <= thresholds, 1); % Find✓
first threshold above dominant frequency
if isempty(class_idx)
    class_idx = length(classes); % Assign highest class if✓
frequency exceeds all thresholds
end
vibration_class = classes{class_idx};

% Display Results in MATLAB Command Window
fprintf('Dominant Frequency: %.2f Hz\n', dominant_freq);
fprintf('Vibration Level: %s\n', vibration_class);

% Step 8: Plot Graphs for Each Processing Stage and FFT✓
Analysis
figure;

% Plot 1: Raw Input Signal
subplot(3,2,1);
plot(t, input_signal, 'b');
xlabel('Time (s)');
ylabel('Amplitude');
title('Input Signal (FPGA Acquisition)');
grid on;
```

```
% Plot 2: Signal After Notch Filter
```

```
subplot(3,2,2);  
plot(t, signal_notched, 'r');  
xlabel('Time (s)');  
ylabel('Amplitude');  
title('Notch Filter Output');  
grid on;
```

```
% Plot 3: Signal After High-Pass Filter
```

```
subplot(3,2,3);  
plot(t, signal_hp, 'm');  
xlabel('Time (s)');  
ylabel('Amplitude');  
title('High-Pass Filter Output');  
grid on;
```

```
% Plot 4: Smoothed Signal (After Moving Average Filter)
```

```
subplot(3,2,4);  
plot(t, signal_smoothed, 'k');  
xlabel('Time (s)');  
ylabel('Amplitude');  
title('Smoothed Signal (Moving Average)');  
grid on;
```

```
% Plot 5: FFT Power Spectrum and Dominant Frequency Marker
```

```
subplot(3,2,5);  
% Plot the power spectrum up to the Nyquist frequency  
plot(f(1:floor(N/2)), power(1:floor(N/2)), 'b');  
xlabel('Frequency (Hz)');  
ylabel('Power');  
title('FFT Power Spectrum');  
grid on;  
hold on;
```

```
% Mark the dominant frequency on the power spectrum
plot(dominant_freq, power(index), 'ro', 'MarkerSize', 10, ✓
'LineWidth', 2);
text(dominant_freq, power(index), sprintf(' %.2f Hz', ✓
dominant_freq), 'VerticalAlignment', 'bottom');
hold off;

% Add a super-title to the figure for overall context.
sgtitle('Signal Processing: FPGA Acquisition, Filtering ✓
Stages, and FFT Analysis');
```